A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

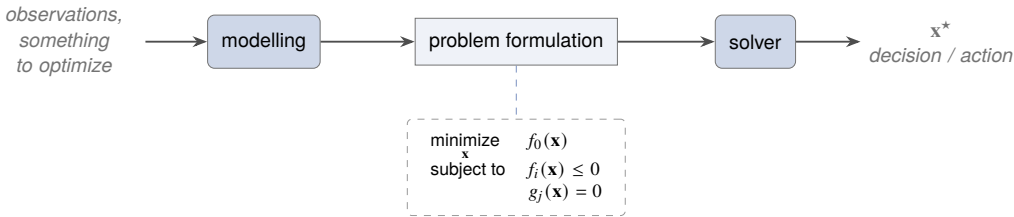
William Zijie Zhang

Master's Thesis Defence
October 2, 2026

Research supervisor: Antoine Lesage-Landry
Department of Electrical Engineering, Polytechnique Montréal

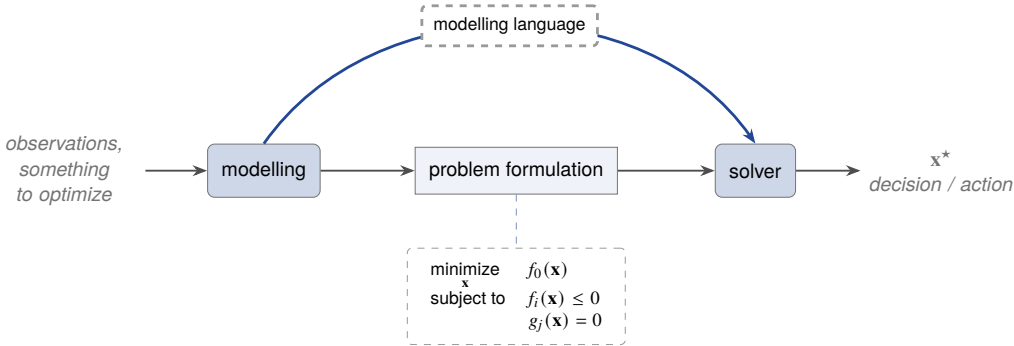
Jury: Sébastien Le Digabel (president) and Antoine Legrain (member)

Optimization in practice



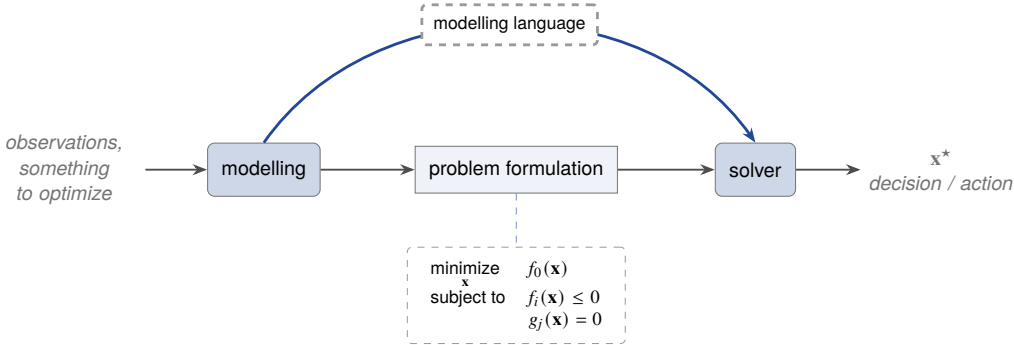
- operate a grid, control a car or a plane, manage a portfolio, design a circuit;
- the formulation says what you want, not how to compute it;

Optimization in practice



- operate a grid, control a car or a plane, manage a portfolio, design a circuit;
- the formulation says what you want, not how to compute it;
- the modelling language is the bridge between the formulation and the solver;

Optimization in practice



- operate a grid, control a car or a plane, manage a portfolio, design a circuit;
- the formulation says what you want, not how to compute it;
- the modelling language is the bridge between the formulation and the solver;
- not in this talk: how to model a problem in a domain, and how a solver works.

Outline

Introduction

Example

Sparse Differentiation Engine

Results

Conclusion

Convex optimization

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0 \\ & g_j(\mathbf{x}) = 0. \end{aligned}$$

- variable $\mathbf{x} \in \mathbb{R}^n$;
- the equality constraints g_j are affine, and f_0 and the f_i are convex;
- why care about convexity?

Boyd and Vandenberghe (2004).

Convex optimization

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0 \\ & g_j(\mathbf{x}) = 0. \end{aligned}$$

- variable $\mathbf{x} \in \mathbb{R}^n$;
- the equality constraints g_j are affine, and f_0 and the f_i are convex;
- why care about convexity?
  - classic answer: a local optimum is a global optimum;
  - modern answer: very actionable, because of reliable software.

Boyd and Vandenberghe (2004).

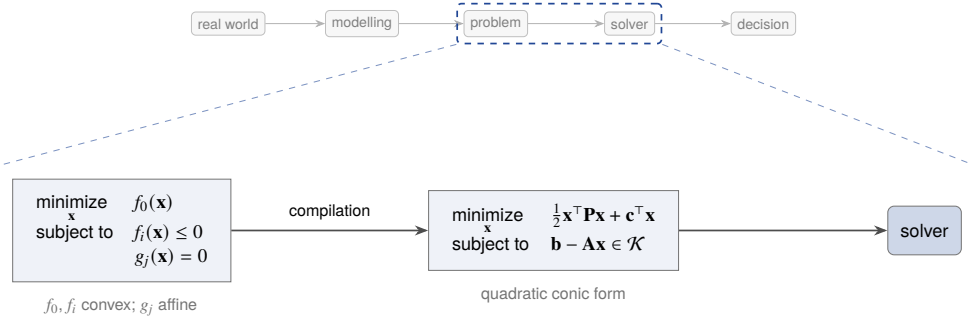
Conic form

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \frac{1}{2}\mathbf{x}^\top\mathbf{P}\mathbf{x} + \mathbf{c}^\top\mathbf{x} \\ \text{subject to} & \mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K}. \end{array}$$

- problem data: $\mathbf{P} \in \mathbb{S}^n_+$, $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$;
- $\mathcal{K} \subseteq \mathbb{R}^m$ is a product of standard cones;
- all general-purpose solvers target this standard form.

Ben-Tal and Nemirovski (2001).

Convex optimization in practice



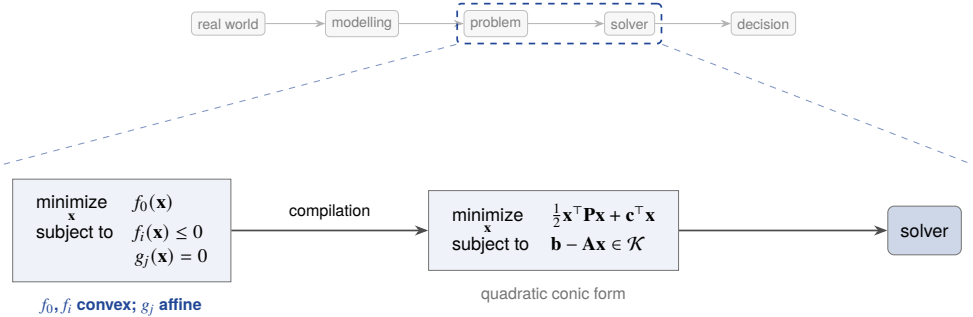
algorithms

interior point, simplex (LP), operator splitting, active set (QP).

solvers

commercial: `Gurobi`, `MOSEK`, `COPT`
open source: `Clarabel`, `SCS`, `HiGHS`

Convex optimization in practice



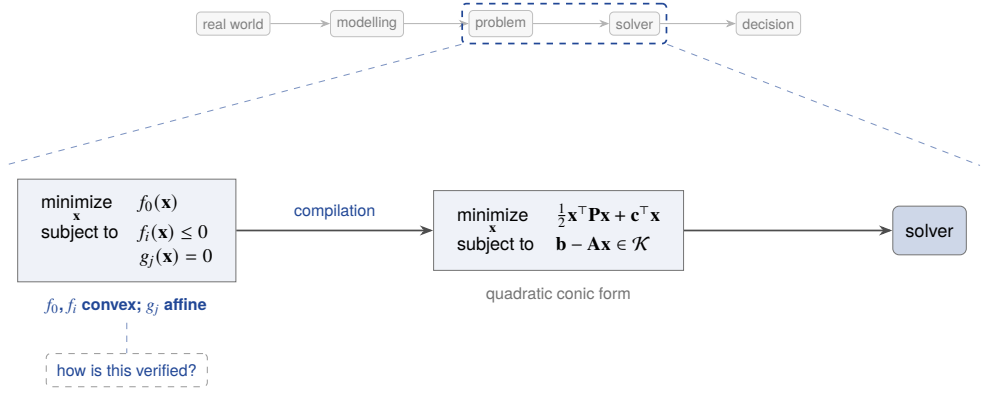
algorithms

interior point, simplex (LP), operator splitting, active set (QP).

solvers

commercial: `Gurobi`, `MOSEK`, `COPT`
open source: `Clarabel`, `SCS`, `HiGHS`

Convex optimization in practice



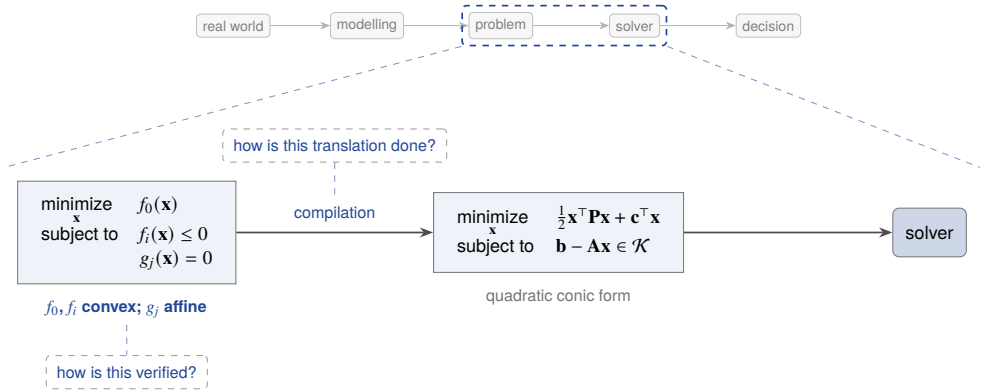
algorithms

interior point, simplex (LP), operator splitting, active set (QP).

solvers

commercial: `Gurobi`, `MOSEK`, `COPT`
open source: `Clarabel`, `SCS`, `HiGHS`

Convex optimization in practice



algorithms

interior point, operator splitting,
simplex (LP), active set (QP).

solvers

commercial: `Gurobi`, `MOSEK`, `COPT`
open source: `Clarabel`, `SCS`, `HiGHS`

Disciplined convex programming

- a grammar for specifying convex optimization problems;
- convex and concave functions are built by combining a set of **atoms**;
- each atom has known curvature, monotonicity, and sign;
- simple rules specify how atoms can be composed.

Grant, Boyd and Ye (2006).

Disciplined convex programming

- a grammar for specifying convex optimization problems;
- convex and concave functions are built by combining a set of **atoms**;
- each atom has known curvature, monotonicity, and sign;
- simple rules specify how atoms can be composed.

- a problem that follows the DCP rules is convex by construction;
- and can be solved globally and efficiently — in theory and in practice.

Grant, Boyd and Ye (2006).

DCP-based modelling languages

- `CVX` (MATLAB), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB), `CVXPY` (Python);

Grant and Boyd (2008); Udell et al. (2014); Fu et al. (2020); Löfberg (2004); Diamond and Boyd (2016).

DCP-based modelling languages

- `CVX` (MATLAB), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB), `CVXPY` (Python);
- the two tasks of a DCP-based modelling language:
  - **analysis**: verify that a high-level problem description follows the DCP rules;
  - **synthesis**: build an equivalent problem in conic form, called canonicalization.

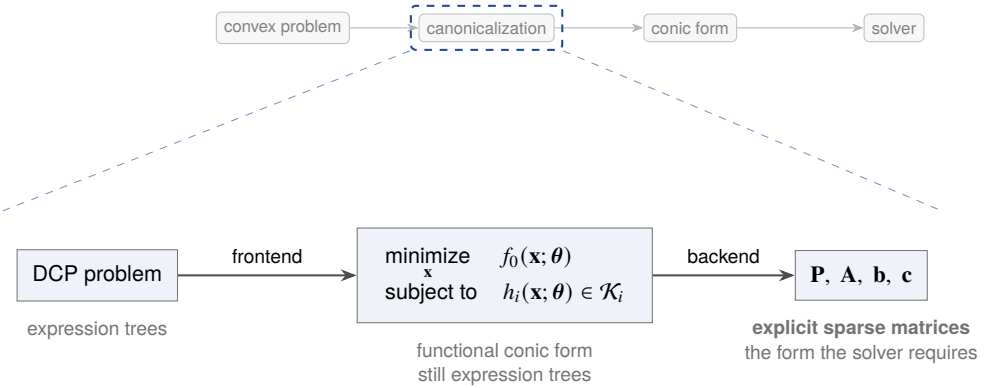
Grant and Boyd (2008); Udell et al. (2014); Fu et al. (2020); Löfberg (2004); Diamond and Boyd (2016).

DCP-based modelling languages

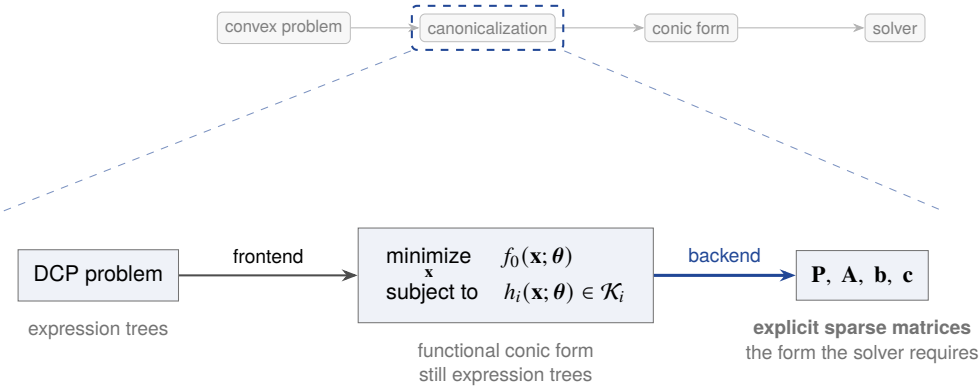
- `CVX` (MATLAB), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB), `CVXPY` (Python);
- the two tasks of a DCP-based modelling language:
  - **analysis**: verify that a high-level problem description follows the DCP rules;
  - **synthesis**: build an equivalent problem in conic form, called canonicalization.

Grant and Boyd (2008); Udell et al. (2014); Fu et al. (2020); Löfberg (2004); Diamond and Boyd (2016).

Canonicalization

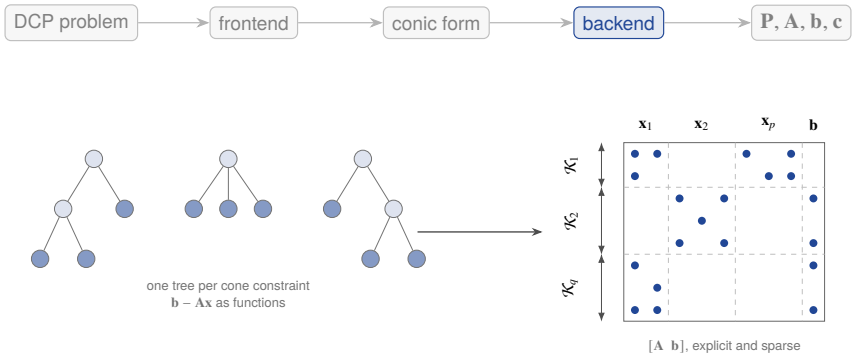


Canonicalization



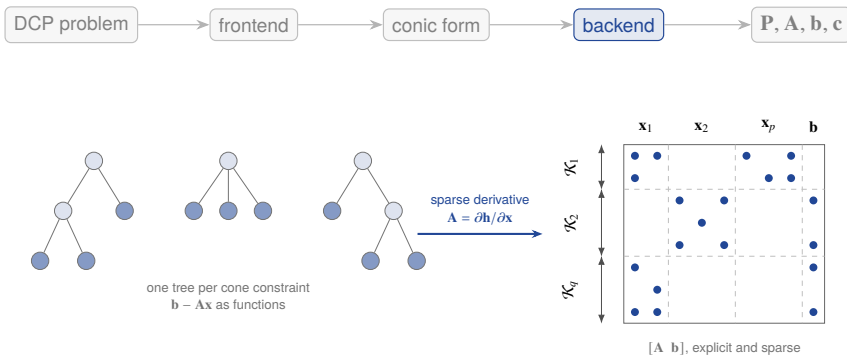
- Our focus is on improving the canonicalization backend of `CVXPY`.

Problem statement



- the expression trees: many different constraints (rows) and variables (columns);
- the matrix $[\mathbf{A}\ \ \mathbf{b}]$: one single sparse constraint matrix with all variables stacked together.

Problem statement



- the expression trees: many different constraints (rows) and variables (columns);
- the matrix $[\mathbf{A} \ \ \mathbf{b}]$: one single sparse constraint matrix with all variables stacked together.

Overview of the contributions

- `SparseDiffEngine`, a C library that computes sparse Jacobians and Hessians directly from the expression trees, with new matrix representations that keep the dense blocks of the problem data dense;
  - existing AD libraries target dense derivatives, or gradients of scalar functions; the solver needs the explicit sparse matrices;
- a new canonicalization backend for CVXPY which we compare against existing methods.

Outline

Introduction

Example

Sparse Differentiation Engine

Results

Conclusion

Example: fitting with regularization



$$\begin{array}{ll} \text{minimize} & \|\mathbf{F}\mathbf{x} - \mathbf{g}\|_2 + \mu\|\mathbf{x}\|_2 \\ \text{subject to} & \mathbf{x} \geq \mathbf{0}. \end{array}$$

- variable $\mathbf{x} \in \mathbb{R}^n$, data $\mathbf{F} \in \mathbb{R}^{m \times n}$, parameters $\mathbf{g} \in \mathbb{R}^m$ and $\mu \geq 0$;
- similar to least-squares;
- physical constraint such as non-negativity of the solution.

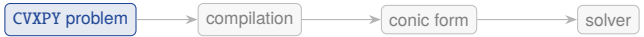
Atoms



Some atoms in the `CVXPY` library:

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`norm`	$\lVert \mathbf{x} \rVert_2$	convex	unknown	nonnegative
`mul`	$\boldsymbol{\mu}\mathbf{x},\ \boldsymbol{\mu} \geq 0$	affine	non-decreasing	any

A CVXPY example




```
import cvxpy as cp
import numpy as np

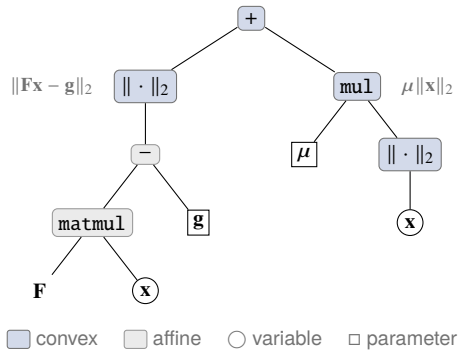
F  = np.random.randn(m, n)
x  = cp.Variable(n)
g  = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)

objective = cp.Minimize(
    cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

Expression tree



`CVXPY` stores the objective and each constraint as an expression tree:



Canonicalization: the frontend



$$\begin{aligned} \underset{t_1, t_2, \mathbf{x}}{\text{minimize}} \quad & t_1 + \mu t_2 \\ \text{subject to} \quad & \mathbf{x} \geq \mathbf{0} \\ & (t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g}) \in \mathcal{K}_{\text{soc},\, m+1} \\ & (t_2,\ \mathbf{x}) \in \mathcal{K}_{\text{soc},\, n+1}. \end{aligned}$$

$$\mathcal{K}_{\text{soc},\, k} = \{(u, \mathbf{v}) \in \mathbb{R} \times \mathbb{R}^{k-1} : \|\mathbf{v}\|_2 \leq u\}, \qquad \mathcal{K} = \mathbb{R}^n_+ \times \mathcal{K}_{\text{soc},\, m+1} \times \mathcal{K}_{\text{soc},\, n+1}$$

- the nonlinear norms are replaced by new variables t_1, t_2;
- every constraint is an affine function belonging to a cone.

Canonicalization: the backend



The conic form needs one stacked variable, $\tilde{\mathbf{x}} = (t_1, t_2, \mathbf{x})$:

$$\begin{array}{ll} \text{minimize} & t_1 + \mu t_2 \qquad\qquad = \mathbf{c}^\top \tilde{\mathbf{x}} \\ \text{subject to} & (\mathbf{x},\ t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g},\ t_2,\ \mathbf{x}) = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}} \in \mathcal{K} \end{array}$$

$$\mathbf{c}^\top = \begin{array}{c} \begin{matrix} t_1 & t_2 & \mathbf{x} \end{matrix} \\ \begin{bmatrix} 1 & \mu & \mathbf{0} \end{bmatrix} \end{array} \qquad [\mathbf{A}\ \ \mathbf{b}] = \begin{array}{c} \begin{matrix} t_1 & t_2 & \mathbf{x} & \mathbf{b} \end{matrix} \\ \begin{bmatrix} \mathbf{0} & \mathbf{0} & -\mathbf{I} & \mathbf{0} \\ -1 & 0 & \mathbf{0} & 0 \\ \mathbf{0} & \mathbf{0} & -\mathbf{F} & -\mathbf{g} \\ 0 & -1 & \mathbf{0} & 0 \\ \mathbf{0} & \mathbf{0} & -\mathbf{I} & \mathbf{0} \end{bmatrix} \end{array}$$

Parameters in CVXPY



Solving the same problem for different values of the parameters:

```
g  = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
mus = [1, 10, 25, 100]

for _ in range(trials):
    g.value = np.random.randn(m)

    for mu_value in mus:
        mu.value = mu_value
        prob.solve()
```

- CVXPY can reuse the problem instance for different parameter values.

Parameters in the CVXPY backend



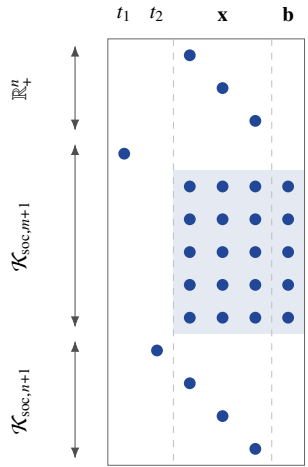
$$\begin{aligned} &\text{minimize} && t_1 + \mu\, t_2 && = \mathbf{c}^\top \tilde{\mathbf{x}} \\ &\text{subject to} && (\mathbf{x},\ t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g},\ t_2,\ \mathbf{x}) && = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}} \in \mathcal{K} \end{aligned}$$

$$\mathbf{c}^\top = \begin{matrix} t_1 & t_2 & \mathbf{x} \end{matrix} \begin{bmatrix} 1 & \mu & \mathbf{0} \end{bmatrix} \qquad [\mathbf{A}\ \ \mathbf{b}] = \begin{matrix} t_1 \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} t_2 \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \mathbf{x} \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \mathbf{b} \\ \\ \\ \\ \\ \end{matrix} \begin{bmatrix} \mathbf{0} & \mathbf{0} & -\mathbf{I} & \mathbf{0} \\ -1 & 0 & \mathbf{0} & 0 \\ \mathbf{0} & \mathbf{0} & -\mathbf{F} & -\mathbf{g} \\ 0 & -1 & \mathbf{0} & 0 \\ \mathbf{0} & \mathbf{0} & -\mathbf{I} & \mathbf{0} \end{bmatrix}$$

- the structure of the problem stays fixed: a zero stays a zero;
- most of the problem data is constant.

Structure in the problem data

convex problem → compilation → conic form → solver



$[\mathbf{A}\ \ \mathbf{b}]$ at $n = 3$, $m = 5$

- most optimization problems have similar structure;
- exploiting structure while computing a sparse derivative is not straightforward.

Outline

Introduction

Example

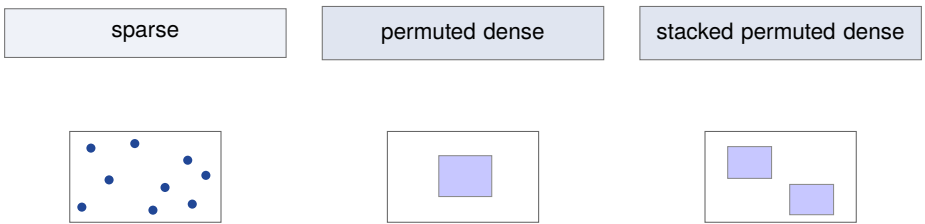
Sparse Differentiation Engine

Results

Conclusion

Contribution

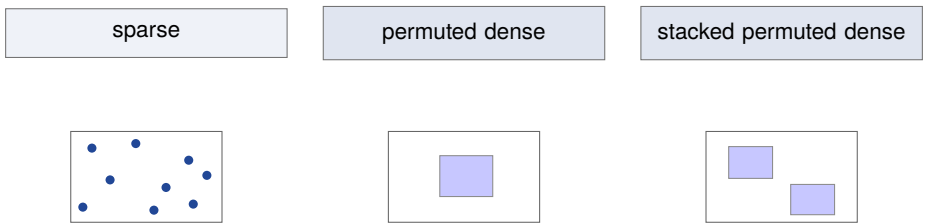
`SparseDiffEngine` is a small C library that computes sparse Jacobians and Hessians from the expression trees of `CVXPY` atoms.



The goal is to use different representations to exploit the different structures in the problem data.

Contribution

`SparseDiffEngine` is a small C library that computes sparse Jacobians and Hessians from the expression trees of `CVXPY` atoms.



The goal is to use different representations to exploit the different structures in the problem data.

- problem level: the structure is fixed and mostly zeros;
- block level: there are sometimes dense blocks which shouldn't be treated as sparse.

Compressed sparse row (CSR) matrix

$$\mathbf{S} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 2 & 0 & 3 & 4 \\ 0 & 5 & 6 & 0 \end{bmatrix}$$

```
int    idxptr[] = {0,  1,          4,      6};
int      cols[] = {1,  0,  2,  3,  1,  2};
double   vals[] = {1,  2,  3,  4,  5,  6};
```

- `CVXPY` stores all problem data as CSR matrices;
- compressed sparse column (CSC) matrices are preferred for column-oriented operations;
- CSR/CSC matrices are the format that most solvers expect.

Davis (2006).

Permuted-Dense matrix

$$\mathbf{A} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & b_{00} & \cdot & \cdot & b_{01} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{10} & \cdot & \cdot & b_{11} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{20} & \cdot & \cdot & b_{21} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{30} & \cdot & \cdot & b_{31} & \cdot & \cdot \end{pmatrix} \qquad \mathbf{X} = \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \\ b_{30} & b_{31} \end{pmatrix}$$

$$\mathbf{r} = (0, 2, 3, 5), \qquad \mathbf{c} = (4, 7), \qquad A_{r_i, c_j} = X_{ij}$$

A permuted dense matrix is a dense submatrix after applying some row and column selections.

Permuted-Dense matrix

$$\mathbf{A} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & b_{00} & \cdot & \cdot & b_{01} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{10} & \cdot & \cdot & b_{11} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{20} & \cdot & \cdot & b_{21} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{30} & \cdot & \cdot & b_{31} & \cdot & \cdot \end{pmatrix} \qquad \mathbf{X} = \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \\ b_{30} & b_{31} \end{pmatrix}$$

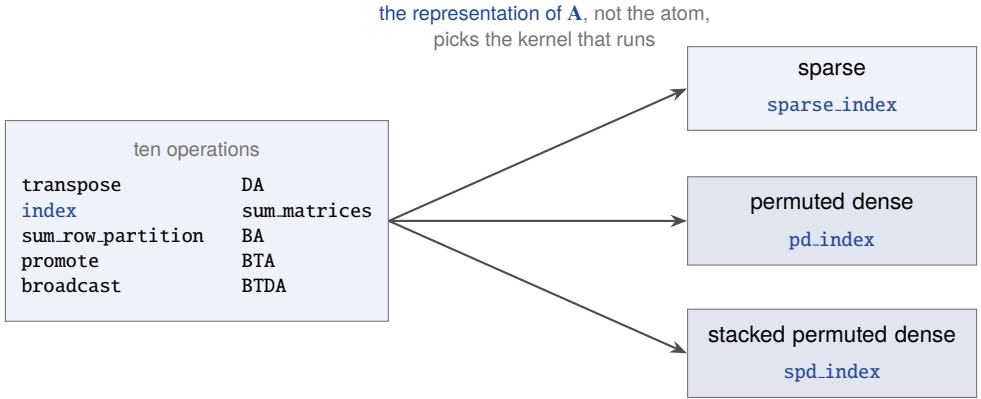
$$\mathbf{r} = (0, 2, 3, 5), \qquad \mathbf{c} = (4, 7), \qquad A_{r_i, c_j} = X_{ij}$$

A permuted dense matrix is a dense submatrix after applying some row and column selections.

- unlocks dense linear algebra storage and operations;
- BLAS: basic linear algebra subprograms are highly optimized dense operations;
- the stacked version can exploit different blocks at disjoint rows and columns.

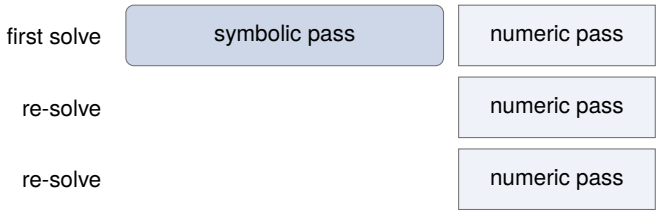
Lawson et al. (1979); Dongarra et al. (1990).

Algorithms



- $\mathbf{A}$ is a Jacobian; $\mathbf{B}$ and $\mathbf{D} = \operatorname{diag}(\mathbf{d})$ are problem data;
- ten affine operations cover most of the atom library;
- each operation has a corresponding kernel for every matrix representation.

Canonicalization in two phases

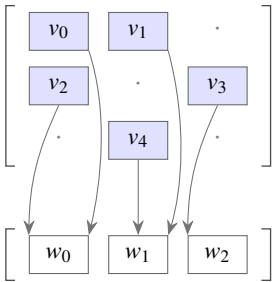


- re-solving a parametrized problem costs one numeric pass.

What the symbolic pass stores

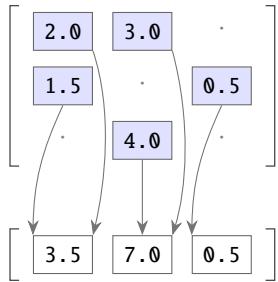
Depending on the kernel, the stored structure is a **sparsity pattern**, a **block layout**, or an **index map**.

symbolic pass, once



`map` $= (0, 1, 0, 2, 1)$

numeric pass, per evaluation



`out[map[`j`]] += vals[`j`]`

The `sum` atom: the symbolic pass records where each nonzero lands, the numeric pass follows the fixed arrows.

`index`, `transpose` and `broadcast` build the same map, with a copy in place of an addition.

Outline

Introduction

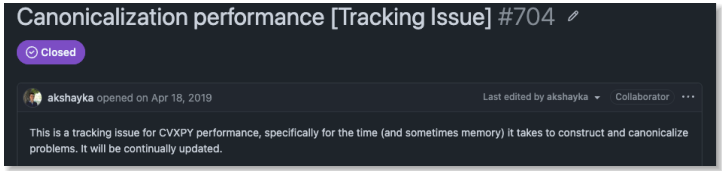
Example

Sparse Differentiation Engine

Results

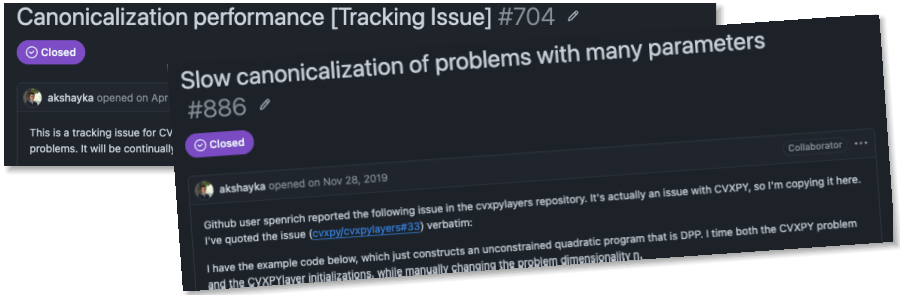
Conclusion

Why do we care about canonicalization?



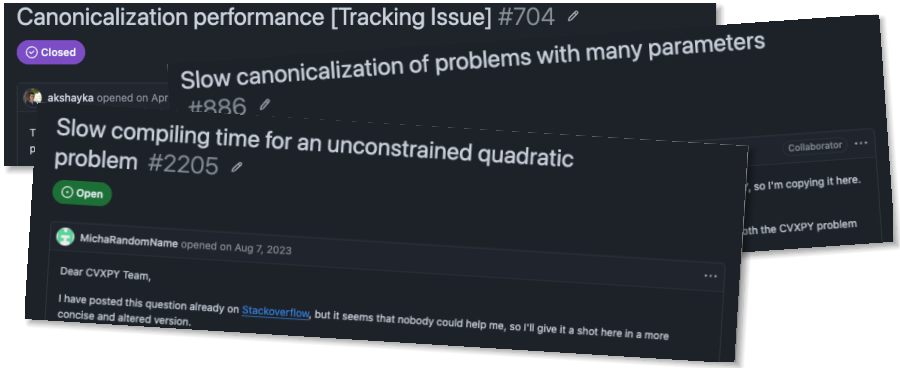
`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



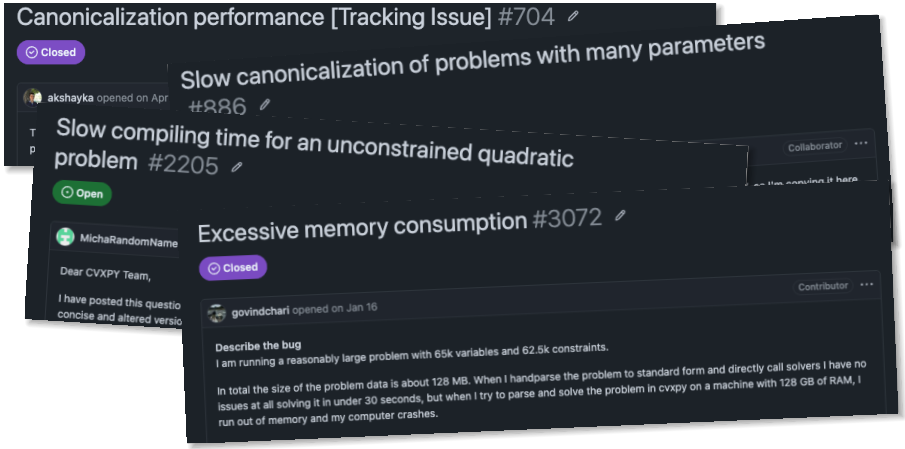
`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



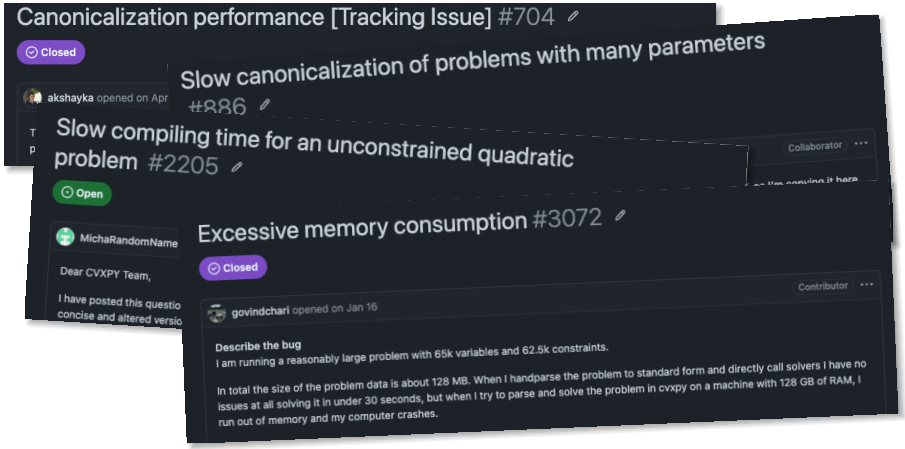
`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



How well does our approach improve the performance of canonicalization?

`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

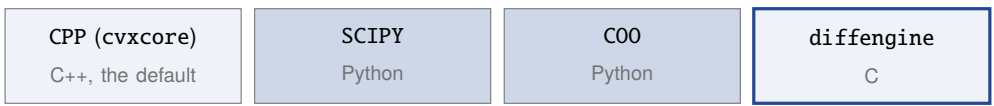
Benchmark setup

The 25 problems of CVXPY's public benchmark suite: portfolio selection, regression and classification, image inpainting, and many more applications of convex optimization.

```
data, _, _ = problem.get_problem_data(
    solver=cp.CLARABEL,
    canon_backend="DIFFENGINE",   # CPP | SCIPY | COO
    ignore_dpp=True)              # no DPP cache: full canonicalization
```

`ignore_dpp=True`: parameters take their current values, so all four backends do the same job, a full canonicalization.

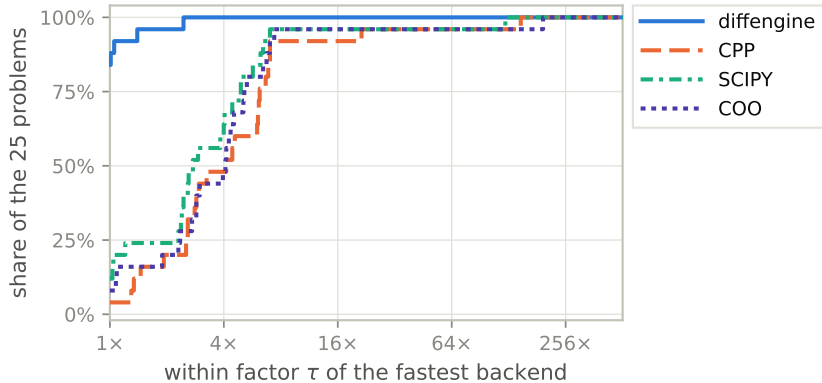
CVXPY's backends:



Single-threaded, 2018 MacBook Pro / Core i7-8850H • 20-minute limit • CVXPY benchmarks (2025), `github.com/cvxpy/benchmarks`.

Performance profile

Share of the 25 problems on which a backend runs within a factor τ of the fastest backend on that problem:

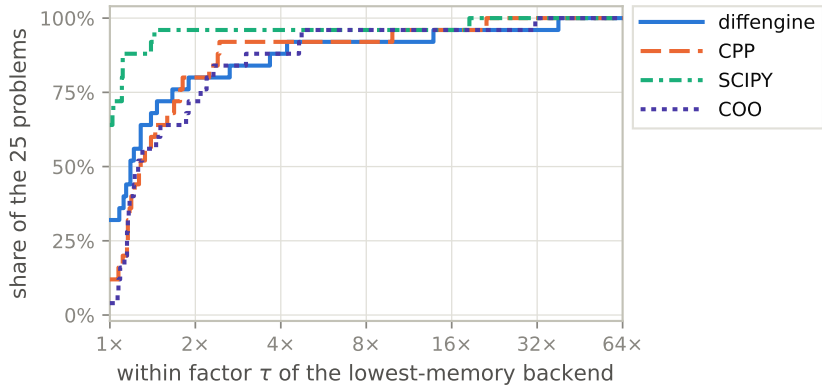


`diffengine` is within 3× of the best backend on *every* problem.

Fastest of the four on 21 of the 25, and more than twice as fast as the best `CVXPY` backend on 17.

Peak memory

The same profile with $t_{p,s}$ replaced by the growth of the process's peak resident memory during canonicalization:



SCIPY has the lowest peak memory on 16 of the 25 problems.

diffengine matches CPP and COO in geometric mean, and uses about 1.4× the memory of SCIPY.

Parametric problems

The seven parametric problems of the suite • times in seconds • re-solve: CVXPY's cached tensor against one numeric pass

	first canonicalization		re-solve	
Benchmark	best CVXPY	diffengine	cached tensor	diffengine
ParamSmallMatrixStuffing	0.40	**0.21**	0.005	**0.001**
ParamConeMatrixStuffing	0.26	**0.14**	0.003	**0.001**
SimpleFullyParametrizedLP	2.17	**0.54**	0.261	**0.126**
ParametrizedQP	10.87	**2.53**	1.485	**0.985**
SimpleScalarParametrizedLP	4.15	**1.03**	0.304	**0.247**
SVMWithL1Regularization	8.21	**1.86**	**0.252**	0.496
FactorCovarianceModel	4.15	**0.86**	**0.139**	0.279

Fastest first canonicalization on every problem; re-solve within about $2\times$ the tensor.

The last two: data almost all constant, and an update still recomputes it.

Outline

Introduction

Example

Sparse Differentiation Engine

Results

Conclusion

Takeaways and conclusions

Summary:

- dense blocks: new matrix representations;
- parametric problems: a symbolic and numeric split;
- a `diffengine` backend for `CVXPY`, fastest of the four on 21 of the 25 problems.

Suggestions from my experience:

- use a `cp.Parameter` when the same problem is solved again, it can really help;
- always try to vectorize your code instead of using loops;
- help the modelling language help you, use NumPy for dense data and SciPy for sparse ones.

Thank you for listening!

References

Agarwal, S., Mierle, K. and the Ceres Solver Team (2023). Ceres Solver. `https://github.com/ceres-solver/ceres-solver`

Agrawal, A., Amos, B., Barratt, S., Boyd, S., Diamond, S. and Kolter, Z. (2019). Differentiable convex optimization layers. *Advances in Neural Information Processing Systems*, 32.

Ben-Tal, A. and Nemirovski, A. (2001). *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*. SIAM.

Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Cederberg, D., Zhang, W., Nobel, P. and Boyd, S. (2026). Disciplined nonlinear programming. arXiv:2606.02896.

Chen, Y., Davis, T. A., Hager, W. W. and Rajamanickam, S. (2008). Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software*, 35(3), art. no. 22.

The Convex.jl developers (2024). Convex.jl release notes, v0.16.0. `https://jump.dev/Convex.jl/stable/release_notes/`

The CVXPY developers (2025). CVXPY benchmarks. `https://github.com/cvxpy/benchmarks`

References (continued)

Davis, T. A. (2006). *Direct Methods for Sparse Linear Systems*. SIAM.

Diamond, S. and Boyd, S. (2016). CVXPY: a Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5.

Diamond, S. and Boyd, S. (2016). Matrix-free convex optimization modeling. In *Optimization and its Applications in Control and Data Sciences*, Springer Optimization and Its Applications 115, pages 221–264. Springer.

Dolan, E. D. and Moré, J. J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213.

Dongarra, J. J., Du Croz, J., Hammarling, S. and Duff, I. S. (1990). A set of Level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, 16(1):1–17.

Fu, A., Narasimhan, B. and Boyd, S. (2020). CVXR: an R package for disciplined convex optimization. *Journal of Statistical Software*, 94(14).

George, A. and Liu, J. W. H. (1981). *Computer Solution of Large Sparse Positive Definite Systems*. Prentice-Hall.

Grant, M. and Boyd, S. (2008). Graph implementations for nonsmooth convex programs. In *Recent Advances in Learning and Control*, Lecture Notes in Control and Information Sciences 371. Springer.

References (continued)

Grant, M., Boyd, S. and Ye, Y. (2006). Disciplined convex programming. In *Global Optimization: From Theory to Implementation*, pages 155–210. Springer.

Gustavson, F. G. (1978). Two fast algorithms for sparse matrices: multiplication and permuted transposition. *ACM Transactions on Mathematical Software*, 4(3):250–269.

Lawson, C. L., Hanson, R. J., Kincaid, D. R. and Krogh, F. T. (1979). Basic linear algebra subprograms for Fortran usage. *ACM Transactions on Mathematical Software*, 5(3):308–323.

Löfberg, J. (2004). YALMIP: a toolbox for modeling and optimization in MATLAB. *IEEE International Conference on Robotics and Automation*, pages 284–289.

Mattingley, J. and Boyd, S. (2012). CVXGEN: a code generator for embedded convex optimization. *Optimization and Engineering*, 13(1):1–27.

Miller, J., Quigley, P. and Zhu, J. (2015). CVXcanon: automatic canonicalization of disciplined convex programs. EE364B project report, Stanford University.

Schaller, M., Banjac, G., Diamond, S., Agrawal, A., Stellato, B. and Boyd, S. (2022). Embedded code generation with CVXPY. *IEEE Control Systems Letters*, 6:2653–2658.

Udell, M., Mohan, K., Zeng, D., Hong, J., Diamond, S. and Boyd, S. (2014). Convex optimization in Julia. *First Workshop for High Performance Technical Computing in Dynamic Languages*, pages 18–28.

Backup slides

Background

The DCP rules
Sufficient, not necessary
Why `CVXPY`
Disciplined parametrized programming
The parameter maps

Positioning and scope

Two ways to get the coefficients
The research gap
Why not matrix-free?
Other answers to the same question
What is borrowed, what is new
`SparseDiffEngine` and DNLP

The engine

When the variable is a matrix
The ten kernels
Which kernel runs on which pair
An elementary cost count
Addition is where the dense path stops
What the two-phase split costs

References

References

Results

How to read a performance profile
The per-problem times
Where `diffengine` is not faster
SDPSegfault1132: the code
Where the time goes on SDPSegfault1132
ConvexPlasticity: a diagonal stored as dense
Instance sizes
Run-to-run variation
Caching without DPP
Isolating the dense representations

The DCP rules

Every atom declares its curvature, its monotonicity in each argument, and the sign of its output:

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`norm`	$\lVert \mathbf{x} \rVert_2$	convex	unknown	nonnegative

The composition $\phi(u_1(\mathbf{x}), \ldots, u_k(\mathbf{x}))$ is convex when ϕ is convex and, for each argument i, one of the following holds:

- ϕ is non-decreasing in i and u_i is convex;
- ϕ is non-increasing in i and u_i is concave; or
- u_i is affine.

Swap convex and concave for the concave rule. A problem is DCP-compliant when the objective and every constraint come out compatible: convex objective, convex $\leq$ concave, affine equalities.

The rules are sufficient for convexity, not necessary: a genuinely convex problem written in an unrecognized form is rejected.

Sufficient, not necessary

The same function, written two ways. Only one of them compiles.

$$\sqrt{x^2 + 1} \;=\; \left\|(x,\ 1)\right\|_2$$

```
cp.sqrt(cp.square(x) + 1)
```

rejected

- `sqrt` is concave and non-decreasing
- its argument $x^2 + 1$ is convex, not affine
- no rule covers concave ∘ convex, so the curvature is unknown

```
cp.norm(cp.hstack([x, 1]))
```

accepted

- `norm` is convex
- its argument $(x, 1)$ is affine
- convex ∘ affine is convex, and the epigraph is $(t,\ x,\ 1) \in \mathcal{K}_{\text{soc},3}$

The rules are a test on the expression, not on the function. Rewriting is the user's job; the atom library is what makes it possible.

Why CVXPY

- open source, embedded in Python, the most widely used programming language
- in development for over ten years, over 6 million downloads a month from PyPI
- 4.5 k citations of the 2016 CVXPY paper

TIOBE index, pypistats.org, github.com/cvxpy/cvxpy and Google Scholar, September 2026.

Disciplined parametrized programming

DPP is a second grammar, on top of DCP, restricting how parameters may enter an expression: every parameter is treated as affine, and products of two parametrized expressions are forbidden.

The payoff is that the problem data becomes affine in the parameters:

$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \operatorname{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}$$

- `CVXPY` builds $\mathbf{Q}$ and $\mathbf{R}$ by adding a third dimension to every coefficient matrix of the pass, indexed by the parameter entries
- after a parameter change, a re-solve costs one sparse matrix-vector product per map: no tree is traversed again
- a problem outside the grammar, or one passing `ignore_dpp=True` (as the main benchmark does), builds no tensor: each parameter leaf is replaced by its current value and the whole canonicalization repeats

The tensor is the baseline the parametric results are measured against.

Agrawal et al. (2019).

The parameter maps

Under DPP the data is affine in the parameters, so the family collapses to two cached maps:

$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \operatorname{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}$$

`CVXPY`'s three backends are three representations of that same tensor:

- `cvxcore` (C++) — one sparse matrix per parameter
- `SCIPY` — the slices stacked into one strided sparse matrix
- `COO` — never formed: triples with a parameter index

Two ways to get the coefficients

Every modelling language reviewed produces the same output, the solver problem data. They split into two families by what they ask of the user.

Differentiate an expression graph

- `CasADi`: built for nonlinear optimization; its conic interface `qpsol` differentiates the graph through matrix-vector products, so the sparsity pattern must be detected at runtime and compressed by colouring
- `CVXPY`: composes the linear operators of the conic-form trees in one bottom-up pass, which is equivalent to differentiating them; expressions stay vectorized, so no detection step is needed
- `SparseDiffEngine`: differentiates them outright, one Jacobian per node with respect to *all* the variables, so the chain rule is a single matrix product — not a dictionary of blocks keyed by variable

Read the coefficients the user declares

- `AMPL`, `JuMP`: the user writes each objective and constraint as a sum of terms, and every linear and quadratic coefficient is stored as it is typed
- neither has an atom library or a canonicalization step, so a norm is left to the solver (`AMPL`) or written in conic form by hand (`JuMP`); both scalarize

The research gap

System	Expression storage	Coefficients obtained by	Reused across the family	Repeated at every solve
AMPL	scalar	instance generation	nothing, values folded into the instance	.nl write and read
JuMP	scalar	term lists stored by macros	term lists, modified in place	copy_to into the solver
CasADi	scalar or matrix	differentiation of an expression graph	pattern, colouring, derivative function	one sweep per colour
CVXPY backends	vectorized	composition of linear operators	parameter-to-data tensor, DPP problems only	one tensor-vector product, or the whole pass
SparseDiffEngine	vectorized	structured direct differentiation	problem structure, by the symbolic pass	numeric pass

None of the four combines the two-phase split with direct computation of the problem data.

Two phases like CasADi, but the whole coefficient matrix at once instead of matrix-vector products, and dense storage where the structure is dense.

Why not matrix-free?

Diamond and Boyd answer the same question with the opposite design: never form $\mathbf{A}$ at all.

Matrix-free

- every linear function is an abstract operator that only applies $\mathbf{Ax}$ and $\mathbf{A}^\top\mathbf{y}$
- no assembly, so no assembly cost
- serves first-order and Krylov-based solvers, which need nothing more than those products

Explicit matrix

- `Clarabel` and `MOSEK` (interior point), `SCS` and `OSQP` in their default direct mode all factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$
- a factorization needs the entries, so these solvers need $\mathbf{A}$ formed
- this thesis keeps $\mathbf{A}$ explicit and makes it cheaper to form

Matrix-free removes the cost for the solvers that can do without $\mathbf{A}$; the solvers `CVXPY` calls by default cannot.

Diamond and Boyd (2016), matrix-free convex optimization modeling.

Other answers to the same question

What each system does about the cost of forming the problem data:

System		Approach
`CVXGEN`	2012	generates a custom C solver for one family of small QPs, so canonicalization happens once, offline
`CVXcanon`	2015	the same matrix stuffing, moved from Python into C++; renamed `cvxcore`, it is the `CPP` backend of the benchmarks
matrix-free `CVXPY`	2016	never forms $\mathbf{A}$ (previous slide)
`CVXPYgen`	2022	the same idea as `CVXGEN`, for `CVXPY`: code generation for one parametrized problem
`Convex.jl`	2024	internals rewritten on top of JuMP's `MathOptInterface`
`diffengine`		forms $\mathbf{A}$ explicitly and online, with a two-pass engine that keeps the dense blocks dense

Mattingley and Boyd (2012); Miller et al. (2015); Diamond and Boyd (2016); Schaller et al. (2022); the Convex.jl developers (2024).

What is borrowed, what is new

Borrowed

- the symbolic and numeric split: sparse direct solvers analyse a pattern once and refactorize on it for new values (George and Liu, 1981; `CHOLMOD`), and Gustavson's sparse product has the same two phases
- the same split in the Jacobian interfaces of NLP solvers such as `IPOPT`
- dense blocks inside sparse Jacobians: finite-element assembly, and nonlinear least squares in `Ceres`

New here

- permuted dense and stacked permuted dense as the matrix classes of a differentiation engine, with kernels under which a dense block stays dense through every product
- the split applied to the backend of a DCP modelling language: the structure is cached per problem, with no grammar asked of the parameters
- a working `CVXPY` backend, merged on September 9, 2026 as an opt-in for parameter-free problems

The ideas are known; what is new is where they are applied, and the data structures that let them apply.

George and Liu (1981); Chen et al. (2008); Gustavson (1978); Agarwal et al. (2023).

SparseDiffEngine and DNLP

One library, two uses.

- SparseDiffEngine was developed jointly by Daniel Cederberg and the author
- it is also the differentiation engine of disciplined nonlinear programming (DNLP), which uses the atoms canonicalization never reaches: exp, log, entr, ...
- canonicalization only sends it affine atoms and quadratic forms
- this thesis contributes the use of the engine for canonicalization:
  - the CVXPY backend diffengine, merged as an opt-in for parameter-free problems
  - the re-solve path that keeps the symbolic pass for parametric problems, under review (CVXPY pull request #3449)
  - the benchmark study of Chapter 4

Cederberg, Zhang, Nobel and Boyd (2026).

When the variable is a matrix

Multiplying a matrix variable $\mathbf{V} \in \mathbb{R}^{2\times 2}$ by dense $\mathbf{B}$ acts on each column separately: one dense block per column of $\mathbf{V}$, at disjoint rows and different columns.

$$\mathbf{M} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & b_{00} & b_{01} & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{10} & b_{11} & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & b_{00} & b_{01} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & b_{10} & b_{11} & \cdot & \cdot \end{pmatrix}$$

$$\mathbf{X}_1 = \mathbf{X}_2 = \mathbf{B}, \qquad \mathbf{r}_1 = (0,1),\ \mathbf{c}_1 = (4,5), \qquad \mathbf{r}_2 = (2,3),\ \mathbf{c}_2 = (6,7)$$

- no single triple describes this: merging the blocks would store the zeros between them, and the sparse form would lose the dense structure
- a stacked permuted dense matrix is the vertical union of blocks at pairwise disjoint rows — every operation treats them one at a time, and the global matrix is never formed
- the stack owns one value buffer and each block points into it, so $\mathbf{B}$ is stored once however many columns $\mathbf{V}$ has

The ten kernels

Ten operations cover the whole atom library, each implemented once per representation, as a symbolic-pass and a numeric-pass function.

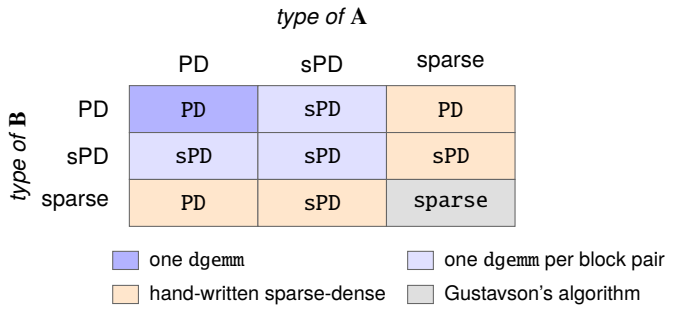
Expression	Kernel
`index`, `transpose`	`index`
`sum`	`sum_row_partition`
`broadcast`, `promote`	`broadcast`, `promote`
`vector_mult`	`DA`
`add`	`sum_matrices`
`left_matmul`	`BA`, `transpose`
`quad_form`	`BTA`, `BTDA`

- no per-atom derivative code: every atom reaches its Jacobian through one of these
- each kernel is written once per representation — `index` copies column indices on CSR, intersects the row vector on a dense block, and drops the empty blocks of a stacked one
- the dense form is absorbing: in a product, a sparse operand never flattens a block

Which kernel runs on which pair

PD is a dense block at permuted positions $(\mathbf{X}, \mathbf{r}, \mathbf{c})$; sPD is the stack of such blocks of the previous slide. Written out, the product of two permuted dense matrices is

$$\mathbf{B}^\top \mathbf{A} = \mathbf{S}_{c_B}^\top \mathbf{X}_B^\top \underbrace{\left(\mathbf{S}_{r_B} \mathbf{S}_{r_A}^\top\right)}_{\text{index-set intersection}} \mathbf{X}_A \mathbf{S}_{c_A}.$$



Eight of the nine kernels come out dense; cost depends on block size, not on m and n.

An elementary cost count

The product $\mathbf{B}^\top\mathbf{A}$ of two permuted dense blocks, $\mathbf{X}_A \in \mathbb{R}^{m_A \times n_A}$ and $\mathbf{X}_B \in \mathbb{R}^{m_B \times n_B}$, with $s = |\mathbf{r}_A \cap \mathbf{r}_B|$ shared rows:

	Symbolic pass	Numeric pass	Indices stored
permuted dense	merge two sorted index vectors, $O(m_A + m_B)$	$n_B\, s\, n_A$ multiply-adds, one `dgemm`	$n_A + n_B$
Gustavson, same blocks	build the full pattern, $O(n_B\, s\, n_A)$	the same $n_B\, s\, n_A$, index-driven loop	$n_A\, n_B$

- a stacked operand: the same counts, summed over the pairs of blocks whose rows intersect
- dense times generic sparse: one multiply-add per stored nonzero of the sparse operand in the shared rows, per column of the dense block

An asymptotic gain in the symbolic pass and the index storage; the same operation count in the numeric pass, where BLAS gains a constant factor.

Addition is where the dense path stops

- the Jacobian of $\mathbf{F}\mathbf{x} + \mathbf{y}$ holds the block of $\mathbf{F}$ at the columns of $\mathbf{x}$ *and* an identity at the columns of $\mathbf{y}$
- no single triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$ covers both, so the sum is performed on CSR views of the two operands instead
- the views are built during the symbolic pass and their value arrays point at the dense blocks, so no values are copied
- the result is a sparse matrix, and so is everything computed from it

Products, by contrast, keep the dense representations: a product with a permuted dense or stacked permuted dense operand gives back a matrix of the same kind.

What the two-phase split costs

- the numeric pass re-evaluates the expression trees at every parameter change, so the `problem` object must stay in memory for the lifetime of the problem family
- the parameter-to-data tensor has no such constraint: once it is built, a re-solve needs only its product with the current parameter values, and the formulation can be set aside
- a parameter update also forces the numeric pass to recompute the entire constant part of the problem data, which is why the two problems with almost-constant data and a few scalar parameters re-solve faster through the tensor

This is the one place the tensor is genuinely the better object.

How to read a performance profile

A performance profile compares the backends over the whole suite at once, rather than one problem at a time.

$$f_s(\tau) = \frac{1}{n} \left| \{ p : t_{p,s} \leq \tau \min_b t_{p,b} \} \right|$$

- $t_{p,s}$ is the time of backend s on problem p, and $\min_b t_{p,b}$ is the time of the fastest backend on that same problem
- $f_s(\tau)$ is the fraction of the n problems on which backend s lands within a factor τ of that best time
- at $\tau = 1$: the fraction of problems on which the backend is outright fastest
- where the curve reaches 1: the factor the backend needs in order to cover every problem in the suite

Dolan and Moré (2002).

The per-problem times

25 problems of CVXPY's public benchmark suite • `get_problem_data` with `Clarabel`, solver excluded • the `ignore_dpp` path • single-threaded, 20-minute limit

Benchmark	CPP (s)	SCIPY (s)	COO (s)	diffengine (s)	Ratio
UnconstrainedQP	10.32	8.52	13.54	**0.07**	0.01
LeastSquares	9.61	10.08	10.68	**1.45**	0.15
SimpleLP	37.57	29.99	30.96	**6.12**	0.20
Cajas	22.67	2.91	2.89	**1.07**	0.37
ConeMatrixStuffing	11.06	9.34	16.99	**3.95**	0.42
	⋮	16 further problems		⋮	
CVaR	35.85	**24.79**	26.86	24.94	1.01
OptimalAdvertising	6.43	2.32	**2.25**	2.36	1.05
SDPSegfault1132	64.84	**9.51**	43.00	13.21	1.39
ConvexPlasticity	**0.35**	**0.35**	**0.35**	0.85	2.48
Geometric mean					**0.36**

Fastest of the four on 21 of the 25 problems, and more than twice as fast as the best CVXPY backend on 17.

Where `diffengine` is not faster

Four of the 25 problems have a ratio above 1:

Problem	Ratio	What dominates	Slide
CVaR	1.01	copying one dense $131\,072 \times 768$ scenario block: no structure to exploit, and `SCIPY` copies it as fast; the ratio was 0.95–1.02 over three days	—
OptimalAdvertising	1.05	a tie with `C00` and `SCIPY` (2.36 s against 2.25 s)	—
SDPSegfault1132	1.39	three dense 49.5 M-entry blocks that cancel only in the final sum	code, profile
ConvexPlasticity	2.48	a diagonal matrix stored as a dense array, taken as a dense $N \times N$ block	mechanism

Close to 1 but faster: Murray (0.96) and SemidefiniteProgramming (0.83).

Two are ties within the run-to-run spread; the other two come from how constant data is handled, not from computing derivatives.

SDPSegfault1132: the code

SDPSegfault1132, where diffengine is 1.39× behind

```
n = 100
V = ...                          # dense, (n, n-1)
D = ...                          # dense, (n, n)
W = np.ones((n, n))              # dense, (n, n)

G = cp.Variable((n-1, n-1), PSD=True)
A = cp.kron(e,   cp.reshape(cp.diag(V @ G @ V.T), (1, n)))
B = cp.kron(e.T, cp.reshape(cp.diag(V @ G @ V.T), (n, 1)))
C = cp.norm(cp.multiply(W, A + B - 2 * V @ G @ V.T - D), p='fro')

prob = cp.Problem(cp.Maximize(cp.trace(G) - C))
```

- every constant is dense, and `V @ G @ V.T` sandwiches the variable between two of them
- diffengine takes 13.21 s against SCIPY's 9.51 s: processing the dense constant data dominates

Where the time goes on SDPSegfault1132

With dense $\mathbf{V} \in \mathbb{R}^{100\times 99}$, the Jacobian of $\mathbf{V}\mathbf{G}\mathbf{V}^\top$ is one dense block of 49.5 M entries, and the problem builds it three times.

`diffengine` (development build)	s
symbolic pass	5.6
numeric pass	7.2
COO to CSR	4.0
cone encoding	2.8
cone formatting	5.5

`SCIPY`	s
CSR products ($\mathbf{V} \otimes \mathbf{V}$)	3.4
Kronecker products	3.0
CSR additions	2.2
rest	0.3

- the sum $\mathbf{A} + \mathbf{B} - 2\mathbf{V}\mathbf{G}\mathbf{V}^\top - \mathbf{D}$ cancels to 1.01 M nonzeros, and neither backend can see this in advance
- `SCIPY`'s CSR addition drops the zeros at the "+"; the engine keeps the 49.5 M-entry pattern, 48.5 M of them exact zeros, until the cone formatting
- on one $\mathbf{V}\mathbf{G}\mathbf{V}^\top$ alone the engine's two passes take 4.1 s, against 15.7 s for `SCIPY`: the loss is in the sum, not in the dense block

cProfile, 2026-08-22; the development build takes 25 s here, the pinned build of the table 13.2 s. Fix, not yet done: drop exact zeros on emission.

ConvexPlasticity: a diagonal stored as dense

```
N = 3000
D = H * np.eye(N)            # diagonal content, dense ndarray
... + 0.5 * cp.quad_form(p - p_old, D)
```

- the quad-form converter chose the dense path by the *container*, not the content, so $\mathbf{D}$ became a dense $N \times N$ Hessian block
- $\mathbf{P}$ stored 9.05 M entries where the true pattern has 33 k: $274\times$ padding, almost all of it explicit zeros; CVXPY's own $\mathbf{P}$ is clean
- on the pinned build: 2.03 s against 0.35 s for all three CVXPY backends, and 1.2 GB more peak memory
- the fix is the density test the matmul converter already had: a constant with under $5\,\%$ nonzeros takes the sparse path, and the time drops to 0.85 s

The three CVXPY backends take the same 0.35 s, so that time is the shared reductions; the 0.5 s diffengine still spends above it has not been profiled yet.

The table's row is from the later build with the fix; the rest of the diffengine column is the pinned build. Revision (vi) re-runs the column on one build.

Instance sizes

$\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{P}$ as `diffengine` emits them • params: the number of parameter objects • ratio as in the per-problem table

Benchmark	m	n	nnz($\mathbf{A}$)	nnz($\mathbf{P}$)	params	ratio
UnconstrainedQP	1 009	289	18.6 k	—	—	0.01
LeastSquares	7 000	9 000	14 M	7 000	—	0.15
Yitzhaki	89.8 k	44.9 k	4.57 M	—	—	0.16
HuberRegression	13.5 k	16.5 k	13.5 M	4 500	—	0.16
SimpleQP	8 020	1 600	12.8 M	2.56 M	—	0.18
FactorCovarianceModel	75.3 k	75.2 k	6.45 M	87.5 k	2	0.20
SimpleLP	20 M	10 M	20 M	—	—	0.20
SVMWithL1Regularization	51 k	26 k	12.6 M	—	1	0.23
SimpleScalarParametrizedLP	4 M	2 M	4 M	—	1	0.25
SimpleFullyParametrizedLP	2 M	1 M	2 M	—	1	0.25
ParametrizedQP	10.8 k	8 400	14.4 M	6 000	2	0.26
TvInpainting	2.61 M	1.05 M	3.63 M	—	—	0.34
Cajas	641 k	2 450	1.96 M	—	—	0.37

Benchmark	m	n	nnz($\mathbf{A}$)	nnz($\mathbf{P}$)	params	ratio
SlowPruning	400 k	402 k	8.4 M	400 k	—	0.39
SmallMatrixStuffing	4 000	4 000	4 000	—	—	0.40
QuantumHilbertMatrix	6 176	6 176	81.1 k	4 096	—	0.41
ConeMatrixStuffing	5 000	5 000	5 000	—	—	0.42
ParamSmallMatrixStuffing	300	300	300	—	3	0.75
ParamConeMatrixStuffing	200	200	200	—	3	0.79
SemidefiniteProgramming	20.2 k	20.1 k	2.43 M	—	—	0.83
Murray	490 k	245 k	1.5 M	—	—	0.96
CVaR	264 k	132 k	25.7 M	—	—	1.01
OptimalAdvertising	252 k	250 k	1 M	—	—	1.05
SDPSegfault1132	15 k	4 951	1.01 M	—	—	1.39
ConvexPlasticity	36 k	33 k	72 k	9.05 M	3	2.48

Run-to-run variation

The printed tables are single runs. The runs that were repeated:

- same day, three runs, the five DPP parametric problems, all backends: 40 of 45 cells spread by less than $9\,\%$; four of the other five are under a second
- three days, the 17 parameter-free problems, with a different engine build each day: times move by up to $40\,\%$, but the ratio to the best `CVXPY` backend moves by at most 0.04 on 13 of the 17
- the ratios that move more are SemidefiniteProgramming (0.55–0.84), SlowPruning (0.31–0.39), SDPSegfault1132 (1.23–1.41) and OptimalAdvertising (1.05–2.57)

Gaps of 3 to $100\times$ are far outside this spread; the ratios near 1 are not, and could go either way.

Revision (iv): five runs of every cell, each in a fresh process, reported as mean ± standard deviation.

Caching without DPP

The numeric pass needs a fixed expression tree and fixed sparsity patterns. It never asks whether the data is affine in the parameters.

```
sigma = cp.Parameter(pos=True)
prob  = cp.Problem(cp.Minimize(cp.norm1(A @ x - b) / sigma
                               + cp.norm1(x)))
prob.is_dpp()      # False: division by a parameter
```

- `CVXPY`: the data is not affine in σ, so no tensor can be built, and every solve repeats the whole canonicalization
- `diffengine`: the tree and the patterns are the same for every σ, so a new value re-runs the numeric pass only

Not yet measured. Revision (iii): this problem and ConvexPlasticity, which `CVXPY`'s QP path treats as non-DPP, against `CVXPY` re-canonicalizing at every solve.

Isolating the dense representations

The benchmark compares whole backends, so the speedup mixes three things: the dense representations, the two-pass split, and the C code of the engine itself.

	dense blocks	two passes	`SparseDiffEngine` code
`diffengine`	✓	✓	✓
`diffengine`, constants stored as CSR	—	✓	✓
`CVXPY` backends	—	—	—

- first row against second: the same passes and the same C code, so the difference is the dense representations alone
- second row against third: what remains is the split and the implementation
- one switch, `DIFFENGINE_NO_DENSE`, stores every constant as CSR; parametric matrices stay dense in both variants

Not yet measured. Revision (ii): both variants on the full suite.